s

Jornada 1: Del hexadecimal al primer firmware

Daniel Rosso - Pablo Madoery

Fundación Fulgor

19 de agosto de 2026

Antes de arrancar

Presentaciones

- ¿Quién sos? ¿De qué Universidad venis?
- ¿Qué te trajo a este curso?
- ¿Programaste en C antes? ¿Trabajaste con hardware digital?

Agenda

1. ¿Qué es una instrucción?
2. Registros
3. Memoria y direccionamiento
4. Del ensamblador al silicio: el lenguaje máquina
5. Formato R-Type: instrucciones registro-registro
6. Formato I-Type: instrucciones con inmediato
7. Formato S-Type: instrucciones de store
8. Formato B-Type: instrucciones de salto condicional
9. Formatos U-Type y J-Type
10. Resumen y ejercicio de decodificación
11. Toolchain: del código C al binario
12. Primer contacto con C bare-metal
13. spike: corriendo (de verdad) lo que decodificamos a mano
14. Hacia dónde vamos

¿Qué es una instrucción?

Una instrucción es la unidad mínima de trabajo que un procesador puede ejecutar.

- **Perspectiva de software:** es un comando en lenguaje ensamblador (ej. `add`, `lw`).
- **Perspectiva de hardware:** es un número binario (código máquina) que activa señales de control.

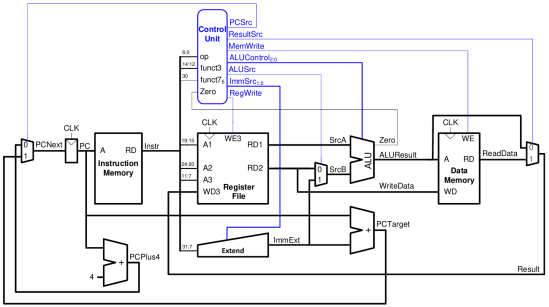
Anatomía básica

`OPCODE | RD | RS1 | RS2 / IMM`

(Operación — Destino — Origen 1 — Origen 2 o Inmediato)

Un vistazo al hardware: el procesador de un ciclo

Antes de entrar en los formatos, conviene ver **dónde** cae cada campo de la instrucción dentro del procesador: decodificar es, literalmente, **configurar este circuito** campo a campo.



- Los campos op/`funct3`/`funct7` entran a la **Control Unit**, que decide qué señales activar: `ALUControl`, `MemWrite`, `RegWrite`, `ResultSrc`/`PCSrc`.
- `rs1`, `rs2`, `rd` son directamente los índices (`A1`, `A2`, `A3`) que direccionan los puertos del **Register File**.
- El inmediato pasa por el bloque **Extend** (según `ImmSrc`) y de ahí entra a la ALU o al cálculo del próximo PC.

Introducción a RISC-V: la ISA abierta

RISC-V es un estándar abierto gestionado por la Fundación RISC-V.

- **RV32I (Base):** el conjunto de instrucciones enteras de 32 bits. Es el mínimo obligatorio para cualquier implementación y permite ejecutar programas en C.
- **Independiente de la implementación:** RISC-V define *qué* hace cada instrucción (el contrato), no *cómo* se construye el hardware por dentro: la misma ISA puede correr en un core chico de 2 etapas o en uno superescalar out-of-order, y ambos son RISC-V válidos.
- **Diseño modular:** a diferencia de x86, RISC-V se expande mediante extensiones opcionales:

Extensión	Funcionalidad
M	Multiplicación y División de hardware.
A	Instrucciones Atómicas (Sincronización - Read/Modify/Write).
F / D	Punto Flotante (Precisión Simple / Doble).
C	*Compressed*: instrucciones de 16 bits para ahorrar espacio.

En este curso trabajaremos con la variante **RV32IM**.

El banco de registros: dónde vive el dato mientras se opera

El procesador no opera directamente sobre la memoria: primero trae los datos a un banco de **32 registros de 32 bits** (`x0` a `x31`), opera sobre ellos, y recién después (si hace falta) escribe el resultado de vuelta a memoria.

- **Load/store**: las instrucciones aritméticas (`add`, `sub`...) solo operan sobre registros, nunca directo sobre memoria: para eso están `lw`/`sw` aparte. Mismo esquema que ARM (familia Cortex); x86 es la excepción, que sí permite un operando de memoria directo en una instrucción aritmética.
- Cada instrucción R/I-type referencia hasta 3 de estos registros: `rs1`, `rs2` (fuentes) y `rd` (destino).
- `x0` es especial: **siempre vale 0**, incluso si se intenta escribir en él (la escritura se descarta).

Fuera de alcance de esta jornada

El contrato de llamada a funciones completo (qué registros preserva cada función, cómo se pasan los argumentos que no entran en `a0`–`a7`...) se ve en detalle cuando empecemos a compilar C real (Jornada 3). La convención de alias (`sp`, `a0`, `t0`...) la vemos ahora mismo, en la próxima slide.

Convención de nombres (ABI): alias y función de cada registro

ABI = *Application Binary Interface*: la convención que define qué alias y qué rol tiene cada registro para que código compilado por separado (funciones, bibliotecas) pueda interoperar.

Registro	Alias (ABI)	Función
x0	zero	Constante 0: leerlo siempre da 0; escribirlo no hace nada
x1	ra	Dirección de retorno (*return address*): la carga jal/jalr
x2	sp	Puntero de pila (*stack pointer*): tope del stack activo
x3	gp	Puntero global: referencia fija a datos globales/estáticos
x4	tp	Puntero de thread: datos propios privados de cada hilo (tp + offset)
x5–x7	t0–t2	Temporales: de uso libre, NO se preservan entre llamadas
x8	s0/fp	Guardado / puntero de marco (*frame pointer*): SÍ se preserva
x9	s1[1]	Guardado: SÍ se preserva entre llamadas
x10–x11	a0–a1	Argumentos 1-2 de la función / valores de retorno
x12–x17	a2–a7	Argumentos 3-8 de la función
x18–x27	s2–s11	Guardados: SÍ se preservan entre llamadas
x28–x31	t3–t6	Temporales: de uso libre, NO se preservan entre llamadas

[1] s0 y s1 (x8–x9) quedan pegados a los argumentos porque la extensión C (comprimida, 16 bits) solo puede nombrar 8 registros (x8–x15, 3 bits de campo): son los únicos *saved registers* alcanzables por una instrucción comprimida, por eso el compilador los prioriza. s2–s11 quedan afuera de ese rango.

Memoria direccionable por bytes

En RISC-V **cada byte** tiene una dirección única. Una palabra (32 bits = 4 bytes) empieza cada 4 direcciones: para acceder al índice de palabra *n* hay que multiplicar por 4.

Dir. base	Datos (Hex)	Palabra
0x0C (12)	CD19A65B	3
0x08 (8)	01EE2842	2
0x04 (4)	F2F1AC07	1
0x00 (0)	ABCDEF78	0

Leer la Palabra 2 (dir. 8):

```
1                lw s3, 8(zero)
2                # s3 = 0x01EE2842
```

Escribir en la Palabra 4 (dir. 16):

```
1                sw t7, 16(zero)
2                sw t7, 0x10(zero)  # equivalente
```

Atención

Leer/escribir una palabra desde una dirección que no es múltiplo de 4 puede generar un error de alineación.

¿Y si el acceso es de 8 bits (`lb`/`sb`)?

`lb`/`lbu` leen **un solo byte**; los otros 3 de esa palabra no se tocan: el resto de los 32 bits de `rd` se completa extendiendo ese byte — con signo (`lb`) o con ceros (`lbu`). `sb` es simétrico: escribe solo el byte direccionado, los otros 3 en memoria quedan igual.

¿Por qué necesitamos el lenguaje máquina?

El **lenguaje ensamblador**: una representación legible por humanos de las instrucciones. Sin embargo, el procesador **no entiende texto**. El hardware solo opera con voltajes: alto (1) y bajo (0).

La Cadena de traducción

`Código C` → `Ensamblador (mnemónicos)` → `Lenguaje Máquina (bin)` → `Silicio (voltajes)`

Ensamblador (legible por humanos):

```
add  s0, s1, s2
lw   t0, 8(sp)
beq  a0, zero, fin
```

Lenguaje máquina (entendido por el hardware):

```
00000001001000001000010000110011
00000000100000010010001010000011
00000000000001010000100001100011
```

Principios de diseño de un ISA

El diseño del formato de instrucción no es arbitrario. RISC-V sigue principios ingenieriles muy claros que surgen de décadas de experiencia en diseño de ISAs:

1. **Regularidad**: todas las instrucciones tienen **exactamente 32 bits**. Esto simplifica el hardware de fetch y decodificación.
2. **Campos en posiciones fijas**: el opcode siempre ocupa los **mismos bits** (6:0). El registro destino (`rd`) siempre está en los bits 11:7. Esto permite decodificación parcial *antes* de conocer el tipo de instrucción.
3. **Inmediatos con extensión de signo**: el bit de signo del inmediato **siempre** está en el bit 31, lo que simplifica el hardware de extensión de signo.
4. **Solo 6 formatos distintos**: a pesar de soportar decenas de instrucciones, RISC-V solo tiene 6 formatos. Cada tipo de instrucción tiene un formato propio: R, I, S, B, U y J.

Contraste con otras ISAs

x86 tiene instrucciones de longitud variable (1 a 15 bytes). Esto ofrece densidad de código mayor, pero requiere un decodificador considerablemente más complejo (y costoso en área y energía).

Mapa general de los 32 bits

En RISC-V, los 32 bits de una instrucción se dividen en **campos**. La posición de cada campo depende del tipo (formato) de la instrucción, pero hay campos que son **siempre iguales**:

Bits 31:25	Bits 24:20	Bits 19:15	Bits 14:12	Bits 11:7	Bits 6:0
`funct7 / imm`	`rs2 / imm`	`rs1`	`funct3`	`rd / imm`	`opcode`
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Campos siempre presentes:

- opcode [6:0]: identifica el *tipo* de operación (load, store, aritmética, branch, etc.)
- `funct3` [14:12]: subclasifica la operación dentro del tipo
- `rs1` [19:15]: primer registro fuente (siempre en esta posición)

Campos que dependen del formato:

- `rd` [11:7]: registro destino (R e I type)
- `rs2` [24:20]: segundo registro fuente (R y S/B type)
- `funct7` [31:25]: extensión del opcode (solo R-type)
- `imm`: inmediato, distribuido según el formato

Formato R-Type: tres registros

El formato **R-type** se usa para instrucciones sobre **tres registros** (dos fuentes, un destino). No tiene inmediato. `add`, `sub`, `and`, `or`, `xor`, `sll`, `srl`, `sra`, `slt`, `sltu` son R-type y comparten opcode=0110011; se distinguen por `funct3`/`funct7` (ej. `add`: funct7=0000000, `sub`: funct7=0100000, etc).

funct7	**rs2**	**rs1**	**funct3**	**rd**	**opcode**
[31:25] 7 bits	[24:20] 5 bits	[19:15] 5 bits	[14:12] 3 bits	[11:7] 5 bits	[6:0] 7 bits

Ejemplo: codificar `add s0, s1, s2`

funct7	rs2	rs1	funct3	rd	opcode
0000000	10010 (s2)	01001 (s1)	000	01000 (s0)	0110011

```
Hexadecimal: 0x01248433
```

Formato I-Type: un registro y una constante

El formato **I-type** se usa cuando una de las fuentes es una **constante embebida** (`addi`, `andi`, `ori`, `lw`, `jalr`...). El inmediato de 12 bits se **extiende en signo** a 32 bits (rango con signo: -2048 a 2047) — el bit 31 de la instrucción es siempre el bit de signo, lo que simplifica el hardware.

imm[11:0]	rs1	funct3	rd	opcode
[31:20] 12 bits	[19:15] 5 bits	[14:12] 3 bits	[11:7] 5 bits	[6:0] 7 bits

Ejemplo: codificar `addi t0, s3, -5` (-5 en complemento a 2, 12 bits: `111111111011`)

imm[11:0]	rs1	funct3	rd	opcode
`111111111011`	`10011 (s3)`	`000`	`00101 (t0)`	`0010011`

```
Hexadecimal: 0xFFB98293
```

Formato S-Type: el inmediato partido

El formato **S-type** (`sw, sh, sb`) no tiene registro destino: necesita **dos** registros fuente (base y dato a guardar), así que el inmediato se parte en dos pedazos para no mover `rs1`/`rs2` de su posición habitual: `imm[11:0] = {inst[31:25], inst[11:7]}`.

imm[11:5]	**rs2**	**rs1**	**funct3**	**imm[4:0]**	**opcode**
[31:25] 7 bits	[24:20] 5 bits	[19:15] 5 bits	[14:12] 3 bits	[11:7] 5 bits	[6:0] 7 bits

Ejemplo: codificar `sw t1, 20(s0)` (offset $20 = 000000010100_2$)

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
0000001	00110 (t1)	01000 (s0)	010	00100	0100011

```
Hexadecimal: 0x00642A23
```

Formato B-Type: saltos relativos al PC

El formato **B-type** se usa para saltos condicionales (`beq, bne, blt, bge, bltu, bgeu`). El destino es **relativo al PC**, y el bit 0 del offset no se codifica (toda instrucción está alineada a 2 bytes, ese bit siempre es 0) — por eso 12 bits codificados dan un rango de 13 bits: -4096 a $+4094$.

imm[12,10:5]	**rs2**	**rs1**	**funct3**	**imm[4:1,11]**	**opcode**
[31:25] 7 bits	[24:20] 5 bits	[19:15] 5 bits	[14:12] 3 bits	[11:7] 5 bits	[6:0] 7 bits

Instr.	**Condición**	**funct3**
`beq`	rs1 == rs2	000
`bne`	rs1 != rs2	001
`blt/bltu`	rs1 $<$ rs2 (con/sin signo)	100 / 110
`bge/bgeu`	rs1 $>=$ rs2 (con/sin signo)	101 / 111

No existen `bgt/ble`: se emulan invirtiendo operandos (`bgt rs1,rs2` $\equiv$ `blt rs2,rs1`).

B-Type: ¿cómo calcular el offset?

Ejemplo: ¿Cuál es el offset de esta instrucción?

```
# PC actual = 0x1000
beq s0, s1, etiqueta   # etiqueta esta en 0x1014
```

1. **Desplazamiento en bytes:** $0x1014 - 0x1000 = 0x14 = 20_{10}$
2. **En 13 bits con signo (bit 0 implícito = 0):** $20_{10} = 0000000010100_2 \rightarrow$ bits 12:1 = 000000001010
3. **Distribuir en los campos:** inst[31]=imm[12]=**0**, inst[30:25]=imm[10:5]=**000000**, inst[11:8]=imm[4:1]=**1010**, inst[7]=imm[11]=**0**

PC-relative

Los saltos B-type nunca usan direcciones absolutas — siempre distancia relativa al PC. Por eso el código es **reubicable**.

Formato U-Type: inmediatos de 20 bits

El formato **U-type** (upper immediate) se usa cuando se necesita un inmediato grande que no cabe en 12 bits. Las instrucciones U-type cargan un valor de 20 bits en los **bits superiores** del registro destino.

imm[31:12]	rd	opcode
[31:12] 20 bits	[11:7] 5 bits	[6:0] 7 bits

LUI — Load Upper Immediate

- `lui rd, imm`
- Carga `imm` en los bits 31:12 de `rd`
- Los bits 11:0 de `rd` se ponen a 0
- opcode = 0110111

AUIPC — Add Upper Immediate to PC

- `auipc rd, imm`
- `rd = PC + (imm << 12)`
- Útil para acceso a datos a distancias grandes del PC (position-independent code)

Construir cualquier constante de 32 bits

Para cargar `0xDEAD0001` en `s0`:

- `lui s0, 0xDEAD0`
  (s0 = 0xDEAD0000)
- `addi s0, s0, 1`
  (bit 11 del inmediato = 0 ⇒ positivo, se suma directo)
- Resultado: 0xDEAD0000 + 0x00000001 = 0xDEAD0001 ✓

Formato J-Type: salto incondicional JAL

El formato **J-type** (Jump) es exclusivo de la instrucción `jal` (Jump And Link). Contiene un inmediato de 21 bits, también relativo al PC, lo que da un rango de salto de ± 1 MB.

imm[20,10:1,11,19:12]	rd	opcode
[31:12] 20 bits	[11:7] 5 bits	[6:0] 7 bits

¿Por qué el inmediato está en ese orden?

Los bits del inmediato J-type están en un orden aparentemente caótico: imm[20,10:1,11,19:12]. La razón es que se diseñó para **maximizar los bits en común con B-type** en las mismas posiciones del registro, especialmente el bit de signo en posición 31. Esto reduce el hardware de extensión de signo.

JAL: Jump And Link

- Guarda `PC+4` en `rd` (dirección de retorno)
- Salta a `PC + offset` (relativo al PC)
- Si `rd = zero`: salto incondicional puro (descarta el retorno)

JALR: Jump And Link Register (I-type)

- `jalr rd, imm(rs1)`
- Salta a `rs1 + imm` (dirección absoluta)
- Para retorno de función: `jalr zero, 0(ra)`

Resumen: los 6 formatos de RISC-V

Formato	Instrucciones	Descripción
R-type	`add, sub, and, or, xor, sll, srl, sra, slt`	Dos registros fuente, un destino. No tiene inmediato.
I-type	`addi, lw, jalr, andi, ori...`	Un registro fuente, inmediato de 12 bits, un destino.
S-type	`sw, sh, sb`	Dos registros fuente, inmediato de 12 bits partido. Sin destino.
B-type	`beq, bne, blt, bge, bltu, bgeu`	Dos registros fuente, offset de 13 bits (PC-relativo). Sin destino.
U-type	`lui, auipc`	Inmediato de 20 bits en bits altos, un destino.
J-type	`jal`	Offset de 21 bits (PC-relativo), guarda PC+4 en rd.

Clave para identificar el formato rápidamente: los 7 bits del opcode (bits 6:0)

0110011=R-type 0010011/0000011=I-type (aritmética/loads) 0100011=S-type 1100011=B-type
0110111/0010111=U-type 1101111/1100111=J-type/`jalr`

Ejercicio: decodificación completa

Decodificar: 0x00C58533

Binario:

```
0000 0000 1100 0101 1000 0101 0011 0011
```

Bits 31:25	Bits 24:20	Bits 19:15	Bits 14:12	Bits 11:7	Bits 6:0
`funct7 / imm`	`rs2 / imm`	`rs1`	`funct3`	`rd / imm`	`opcode`

Clave de opcode (bits 6:0)

0110011=R 0010011/0000011=I 0100011=S 1100011=B 0110111/0010111=U 1101111/1100111=J

- ¿Qué formato es, según el `opcode`?
- ¿Cuánto vale cada campo (`rs1`, `rs2`, `funct3`, `rd`)?
- ¿Qué instrucción en ensamblador es?

Solución: decodificación completa

opcode = `0110011` **→ R-type (registro-registro)**

Extraer campos:

- funct7 = inst[31:25] = `0000000`
- rs2 = inst[24:20] = `01100` = 12 → `a2`
- rs1 = inst[19:15] = `01011` = 11 → `a1`
- funct3 = inst[14:12] = `000` → **add**
- rd = inst[11:7] = `01010` = 10 → `a0`

A diferencia del S-type, acá no hay inmediato que reconstruir: el R-type usa los tres campos de registro completos (`rs1`, `rs2`, `rd`).

Resultado

```
add a0, a1, a2   (a0 = a1 + a2)
```

Ejercicio: decodificar un `sw`

Decodificar: `0x00E7A023`

Binario:

```
0000 0000 1110 0111 1010 0000 0010 0011
```

Bits 31:25	Bits 24:20	Bits 19:15	Bits 14:12	Bits 11:7	Bits 6:0
`funct7 / imm`	`rs2 / imm`	`rs1`	`funct3`	`rd / imm`	`opcode`

Clave de opcode (bits 6:0)

`0110011`=R `0010011/0000011`=I `0100011`=S `1100011`=B `0110111/0010111`=U `1101111/1100111`=J

- Esta vez el opcode da S-type: ¿qué dos campos hay que pegar para reconstruir el inmediato?
- ¿Cuánto vale cada campo (`rs1`, `rs2`, `funct3`)?
- ¿Qué instrucción en ensamblador es?

Solución: decodificar un `sw`

opcode = `0100011` **→ S-type (store)**

Extraer campos:

- imm[11:5] = inst[31:25] = 0000000
- rs2 = inst[24:20] = 01110 = 14 → a4
- rs1 = inst[19:15] = 01111 = 15 → a5
- funct3 = inst[14:12] = 010 → **sw** (word)
- imm[4:0] = inst[11:7] = 00000

A diferencia del R-type, acá no hay `rd`: el inmediato se reconstruye pegando los dos pedazos — `imm = {imm[11:5], imm[4:0]} = 0000000 00000 = 0`.

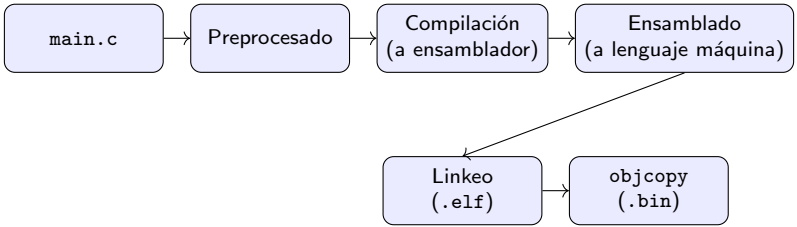
Resultado

`sw a4, 0(a5)` (guarda a4 en la dirección que apunta a5)

Esta es exactamente la instrucción que van a ver ejecutar en `spike` un rato más adelante — ahí a5 va a valer 0x80001000 (la dirección de GPIO) y a4 va a valer 1.

De `.c` a `.bin`: las etapas de la compilación

Todavía no escribimos C — eso arranca en la Jornada 3 — pero ya podemos entender **por qué** un archivo de texto termina siendo la secuencia de bytes que decodificamos recién.



- Cada instrucción de la Sección anterior (`add`, `addi`, `lw`...) es exactamente lo que produce el paso de **ensamblado**.
- El **linkeo** decide en qué dirección de memoria vive cada cosa (lo vemos a fondo en la Jornada 3, con el linker script).
- El `.bin` final es lo que efectivamente vamos a cargar en la BRAM de la FPGA (Jornada 2).

Cross-compiling: compilar para otra arquitectura

Tu PC es x86_64 (o ARM si es un Mac). Si corrés `gcc` a secas, genera código **para tu PC** — no serviría de nada cargado en la FPGA, que corre un procesador **RISC-V**.

Cross-compiling

Compilar en una arquitectura (**host**, tu PC) generando código ejecutable para otra distinta (**target**, el RISC-V de la FPGA). Necesitamos un compilador que corra en el host pero conozca el conjunto de instrucciones del target.

Convención de nombre del toolchain (target triple):

```
riscv-none-elf-gcc
```

- **riscv**: arquitectura destino (lo que decodificamos toda esta jornada)
- **none**: sin sistema operativo — bare-metal (no `linux`, no `none` de escritorio)
- **elf**: formato del binario que produce

Regla de oro

Todas las herramientas de la cadena llevan el mismo prefijo: `riscv-none-elf-gcc`, `riscv-none-elf-objdump`, `riscv-none-elf-objcopy`, `riscv-none-elf-readelf`...

¿Qué significa programar "bare-metal"?

En una PC, un programa en C corre **arriba** de un sistema operativo: pide memoria con `malloc`, escribe con `printf`, y el SO media el acceso al hardware.

En nuestro SoC no hay sistema operativo — el firmware es lo primero y lo único que corre, con acceso directo a los periféricos mapeados en memoria (MMIO, lo vemos en detalle en la Jornada 3):

```
#define GPIO (*(volatile unsigned int *)0x00200000)

void main(void) {
    GPIO = 0x1;   // prende un LED, sin SO de por medio
    while (1) ;
}
```

Dos `*` con roles distintos: el de adentro (`unsigned int *`) es parte del **tipo** — "puntero a"—; el de afuera desreferencia ese puntero para leer/ escribir la dirección como si fuera una variable.

Lo que ya podés reconocer

Ese `GPIO = 0x1` se traduce, tal cual vimos, a una instrucción `sw` (S-type) hacia la dirección `0x00200000`.

¿Cómo sabemos si decodificamos bien?

Recién decodificamos `0x00C58533` a mano y llegamos a `add a0, a1, a2`, y recién escribimos `GPIO = 0x1` sin verlo correr. Confiamos en la tabla de bits — pero **nunca ejecutamos nada**.

spike: el simulador de referencia de RISC-V

`spike` es un simulador funcional (no simula tiempo/ciclos, simula el efecto de cada instrucción) mantenido por la Fundación RISC-V. Es el **modelo de referencia**: cuando alguien diseña un core nuevo, compara su comportamiento contra spike para validar que es correcto.

Nos sirve para lo mismo que a ellos, a otra escala: ejecutar un programa RISC-V real **sin FPGA, sin Vivado, sin esperar síntesis** — perfecto para hoy, que es la única jornada virtual.

Ejercicio: el mismo `GPIO = 0x1`, corriendo en spike

Mismo patrón que recién, con una diferencia: spike reserva 0x0–0x1000 para uso interno, así que en este ejercicio puntual el GPIO queda en otra dirección (0x80001000 en vez de 0x00200000 — lo explicamos en la última slide de esta sección).

```c
#define GPIO (*(volatile unsigned int *)0x80001000)
void main(void) {
    GPIO = 0x1;
    while (1) ;
}
```

```
$ make run
```

```
core   0: 0x80000020 (0x800017b7) lui   a5, 0x80001
core   0: 0x80000024 (0x00100713) li    a4, 1
core   0: 0x80000028 (0x00e7a023) sw    a4, 0(a5)
core   0: 0x8000002c (0x0000006f) j     pc + 0x0
```

Lo que estamos viendo

spike desensambla **cada instrucción retirada**, en vivo. La tercera línea es nuestro `GPIO = 0x1` — comparalo con cómo lo decodificamos a mano en el ejercicio anterior.

spike -d: comandos básicos del debugger

Antes del walkthrough completo, una referencia rápida de los comandos que se van a usar:

- `rs N`: corre `N` instrucciones **en silencio**, sin traza, para saltar rápido a un punto del programa.
- `run N`: corre `N` instrucciones mostrando el desensamblado de cada una, como se vio recién con `make run`, pero paso a paso.
- `reg <hart> <registro>`: lee el valor de un registro (acá siempre `hart 0`, el único core).
- `mem <dirección>`: lee una palabra de memoria en esa dirección.
- `q`: sale del debugger.

Ojo

`reg` no acepta `pc` como nombre de registro; para eso existe el comando separado `pc <hart>`.

El debugger interactivo: antes y después de la escritura

`spike -d` deja tipear comandos e inspeccionar el estado a mano — exactamente como el hex que veníamos decodificando.

```
$ make debug
(spike) rs 14              # correr 14 instrucciones (sin traza)
:
(spike) mem 0x80001000     # ¿qué hay en la dirección de GPIO?
0x00000000
(spike) reg 0 a5           # a5 tiene la dirección (lui a5,0x80001)
0x80001000
(spike) run 1              # una instrucción más: el sw
core   0: 0x80000028 (0x00e7a023) sw a4, 0(a5)
(spike) mem 0x80001000     # ¿y ahora?
0x00000001
(spike) q
```

`mem` pasó de `0x0` a `0x1` — exactamente el efecto que predijimos al decodificar `sw` a mano, pero ahora **lo vimos pasar**.

Nota: la dirección de arranque no es 0x0

Si corrieron `make debug` ya notaron que las primeras 5 instrucciones no son las nuestras:

```
core   0: 0x00001000 (0x00000297) auipc t0, 0x0
core   0: 0x00001004 (0x02028593) addi  a1, t0, 32
...
core   0: 0x80000000 (0x00010117) auipc sp, 0x10   <- acá arranca _start
```

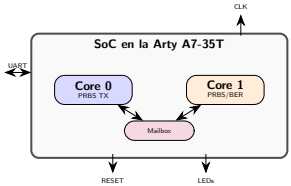
- spike arranca con un **mini trampolín interno** (5 instrucciones) que salta a nuestro `_start`.
- spike también reserva 0x0–0x1000 para su propio debug module — por eso nuestro `link.ld` de hoy usa 0x80000000 (la convención estándar de RISC-V), no 0x000000 como la placa real.
- En la Jornada 2, con PicoRV32 en la FPGA, `_start` va a estar directamente en 0x000000 — ahí no hay trampolín: el reset del core apunta ahí directo.

El SoC que vamos a construir

Hoy decodificamos instrucciones a mano y escribimos nuestro primer `GPIO = 0x1`. En las próximas 4 jornadas ese mismo principio — una instrucción a la vez, un registro MMIO a la vez — se acumula hasta este sistema:

- **Jornada 2:** primer SoC de un core corriendo en la FPGA
- **Jornada 3:** firmware completo (ELF, CRT0, linker, UART)
- **Jornada 4:** segundo core + mailbox compartido
- **Jornada 5:** bootloader — firmware nuevo sin resintetizar

Es un BERT (Bit Error Rate Tester) de comunicaciones: Core 0 genera un stream PRBS con errores inyectados, Core 1 lo verifica y reporta la tasa de error medida.



Todo lo que hay adentro de ese rectángulo lo vamos a entender de punta a punta — porque lo vamos a escribir nosotros.

Más allá del ISA base: se pueden agregar instrucciones

RISC-V no es una lista cerrada: el estándar reserva a propósito un rango de opcode para que cualquiera defina las suyas.

- El mapa de opcodes de 7 bits reserva `custom-0` (`0001011`) y `custom-1` (`0101011`) — entre otros — para instrucciones propias, sin chocar con el resto del ISA.
- Una instrucción custom sigue alguno de los formatos ya vistos (R/I/S/...): mismos campos `funct3`/`funct7` para elegir la operación, mismos `rs1`/`rs2`/`rd`.
- Sirve para acelerar en hardware una operación puntual que en software tomaría varias instrucciones seguidas.

Se puede prototipar en simulador

Antes de tocar hardware real, una instrucción custom se puede agregar al decoder de `spike` — el mismo simulador que usamos hoy — para validar su semántica sin sintetizar nada.

Muchas gracias

Daniel Rosso - Pablo Madoery

rossodaniel@gmail.com